

Posterity - Project Context

What Is Posterity

A social media legacy app that allows users to schedule messages, memories, and content to be delivered to loved ones after death. Think of it as a digital time capsule with a subscription model.

Business Model

- Annual upfront billing (one-off payment per year)
- Multi-year upfront option available (one-off payment) — Phase 4
- Storage fee: \$9.99/year for inactive/skipped years (Horizon tier)

Subscription Tiers

Tier	Price	Messages	Video
Horizon	\$9.99/year	None	None
Basic	\$39/year	4 per year (quarterly)	1 per year
Premium	\$99/year	12 per year (monthly)	4 per year
Legacy	\$299/year	52 per year (weekly)	12 per year
Custom	Contact Us	Custom	Custom
Financial Hardship	Contact Us	1-2 messages	None

All message tiers include optional handwritten note per message.

Horizon Tier - Dual Purpose

1. Inactive account storage - for users with lapsed or paused plans
 2. Legacy builder starter - build account before committing to a full plan
- Content stored but not delivered until a plan is activated
 - Recurring annual subscription that auto-cancels when a full plan is activated
 - Plans upgradeable from within user dashboard

Stripe Products

Product	Price ID
Posterity Horizon	price_1Tcvq1BcdnR2VoDgYustrs4G
Posterity Basic	price_1TeTeyBcdnR2VoDgYhf7uym0
Posterity Premium	price_1Tcvj2BcdnR2VoDgt4oZu8VA
Posterity Legacy	price_1TeTj5BcdnR2VoDgtfA9O3z8

Content Delivery Activation (Death Trigger System)

- Option A: Periodic check-in every 6 months (user confirms alive via email/app)
- Option B: Trusted contact confirms passing
- Grace period TBD (missed check-in, 30 day grace, second notice, delivery triggered)
- Trusted contact needs their own simple portal to confirm and trigger delivery

Tech Stack

Tool	Purpose
Next.js	App framework
Tailwind CSS	Styling
Supabase	Database + Auth
Stripe	Payments
Vercel	Hosting
GitHub	Code storage
Cursor	AI-assisted code editor

Project Location

- Local: C:\Users\jerth\posterity
- GitHub: <https://github.com/jerthwicked/posterity> (PRIVATE)
- Live URL: <https://posterity-seven.vercel.app>

What's Been Built

- Development environment (Node.js v24, Git, Cursor)
- Next.js project created
- Homepage, Login, Signup, Dashboard pages
- Working Supabase authentication
- PWA installed on Android
- Live on Vercel
- MCP servers configured in Cursor (.cursor/mcp.json)
 - GitHub MCP (26 tools, green/enabled)
 - Supabase MCP (enabled via access token)
- Stripe account created (sandbox/test mode)
- 4 Stripe products created (Horizon, Basic, Premium, Legacy)
- Pricing page live at /pricing with all 6 tiers
- Stripe checkout working and tested
- Pricing link in homepage nav

Environment Variables

Set in both .env.local (local) and Vercel (production):

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY

- STRIPE_SECRET_KEY
- NEXT_PUBLIC_SITE_URL=<https://posterity-seven.vercel.app>

MCP Setup

- Config file: C:\Users\jerth\posterity.cursor\mcp.json
- .cursor/mcp.json is in .gitignore (tokens never pushed to GitHub)
- GitHub MCP: uses personal access token, no expiration
- Supabase MCP: uses personal access token (sbp_...)
- Stripe MCP: to be added when Stripe goes live

Session Workflow

- Discuss and plan changes here in Claude.ai first
- Get approval (user types "g") before any code is pushed
- Claude pushes code to Cursor/GitHub directly
- Check live site after changes: <https://posterity-seven.vercel.app>
- Use branches for bigger features (to be implemented soon)
- Claude updates context files at end of every session
- g = go/approved, d = done

What's Next - Phase 3 (Remaining)

- Connect Stripe checkout to Supabase user accounts
- Webhook to handle subscription status changes
- Lock dashboard features behind subscription tiers
- Integrate Claude properly into Cursor workflow

What's Next - Phase 4

- Custom cart system:
 - Plan tier selection
 - Multi-year upfront payment option
 - Automatic storage fee calculation for skipped years
 - Storage fee add-on for inactive accounts
 - Hardship/custom plan flag that routes to rep
- Video upload component in user profiles
- Client-side compression via FFmpeg.wasm
- Trusted contacts system
- Check-in system (every 6 months) with reminder emails
- Death trigger logic
- Message creation (write, schedule, assign recipients)

Video Storage Plan

- Storage via Supabase Storage
- Client-side compression via FFmpeg.wasm (Phase 4)
- Upgrade path: Cloudflare Stream or Mux at scale
- Long term storage goal: up to 10 years
- Storage limits per tier TBD

How To Resume

1. Open Cursor
2. Open terminal (Ctrl + backtick)
3. cd posterity
4. npm run dev
5. Open <http://localhost:3000>

Developer Notes

- Windows 11 machine
- PowerShell terminal (no && between commands)
- Node.js v24.16.0
- Non-technical founder, all code written by AI
- Owner: jerthrwicked (GitHub)
- g = go/approved, d = done
- For file creation use \$content = @" ... "@ then [System.IO.File]::WriteAllText() to avoid UTF-8 corruption